

سوال یک زنجیره مارکوف:

۱. **Markov Property** را تعریف کنید. چرا به آن "بی حافظه" می گویند؟ این ویژگی چگونه بر پیش بینی پذیری سیستم در طول زمان تأثیر می گذارد؟

این ویژگی خاصیت فرایند هایی است که حالت بعدی (next state) صرفاً به همین حالتی که در آن قرار داری وابسته باشد. در اینصورت در هر مرحله بدون توجه به اتفاقاتی که تا لحظه حاضر اتفاق افتاده است سیستم با احتمالاتی که با یک ماتریس توصیف می شود تصمیم به رفتن به حالت بعدی می گیرد. و این عدم توجه به اتفاقات قبلی را بی حافظگی سیستم می گویند.

پاسخ قسمت دوم:

۲. **Transition Matrix** یک Chain Markov ساده با ۲ وضعیت به صورت زیر داریم:

$$P = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix}$$

احتمال توزیع بعد از دو گام را محاسبه کنید، به شرط آنکه سیستم از وضعیت ۱ شروع کند.

این ویژگی باعث می شود تا بدون نیازمندی به اطلاعات گذشته و صرفاً با دانستن حالت فعلی بتوان سیستم را پیش بینی کرد. همچنان می توان با صرفاً ماتریس انتقال آن π ، سیستم را در بلند مدت پیش بینی کرد.

سوال دو :

$$p_1 = p_{11} * p_{11} + p_{12} * p_{21}$$

$$P_2 = p_{12} * p_{22} + p_{11} * p_{12}$$

$$P_1 = 0.7 * 0.7 + 0.3 * 0.4 = 0.61$$

$$P_2 = 0.3 * 0.6 + 0.7 * 0.3 = 0.39$$

$$P_1 + P_2 = 1$$

سوال سه :

۳. توزیع پایا: برای یک زنجیره مارکوف با ۳ وضعیت و ماتریس انتقال زیر:

$$P = \begin{pmatrix} 0.5 & 0.2 & 0.3 \\ 0.3 & 0.4 & 0.3 \\ 0.4 & 0.3 & 0.3 \end{pmatrix}$$

توزیع ایستادن π را با حل معادله $\pi P = \pi$ بیابید.

$$-0.5 * \pi_1 + 0.3 * \pi_2 + 0.4 * \pi_3 = 0$$

$$0.2 * \pi_1 + -0.6 * \pi_2 + 0.3 * \pi_3 = 0$$

$$0.3 * \pi_1 + 0.3 * \pi_2 - 0.7 * \pi_3 = 0$$

$$\begin{pmatrix} -0/5 & 0/3 & 0/4 \\ 0/2 & -0/6 & 0/3 \\ 0/3 & 0/3 & -0/7 \end{pmatrix} \begin{pmatrix} \pi_1 \\ \pi_2 \\ \pi_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

سه بردار ویژه متناظر ماتریس P هم به صورت $(7.04, 4.91, 5.12)$ $(7.07, -7.07, -1.92)$ $(-4.08, -4.08, 8.16)$ به دست می آیند.

که دوتای آخر قابل قبول نیستند چرا که یکی از احتمال ها در هر صورت منفی به دست می آیند. (اگر بردار را در هر عددی ضرب کنیم یکی از مولفه ها منفی خواهد شد.) بنابر این توزیع π مطلوب ما بردار اول یعنی $(7.04, 4.91, 5.12)$ به دست می آید. حال باید آن را نرمال کرد. یعنی جمع سه تا باید یک شود. جمع حال حاضر این بردار برابر است با 17.07 در نتیجه برای به دست آوردن بردار مطلوب π تمام درایه ها را بر این عدد تقسیم می کنیم.

$$\pi = (0.412, 0.288, 0.3)$$

۴. ارگودیک بودن: مفهوم ارگودیک بودن در زنجیره های مارکوف را توضیح دهید. ارگودیک بودن چگونه بر همگرایی زنجیره مارکوف به یک توزیع پایا تأثیر می گذارد؟

یک زنجیره مارکوف ارگودیک است. اگر بتوان با تعدادی حرکت از هر حالتی به حالت دیگر رسید. اگر زنجیره مارکف این خاصیت را نداشته باشد باعث می شود که نتوان توزیع نهایی یا π را حدس زد چرا که در کدام حالت بودن مهم است چون در بعضی از حالت ها دیگر نمی توان به سایر حالت ها رفت. مثلاً اگر ماتریس انتقال به صورت

$$\begin{pmatrix} 1 & 0 & 0 \\ 0/5 & 0/2 & 0/3 \\ 0/7 & 0/2 & 0/1 \end{pmatrix}$$

باشد. در اینجا اگر در هر صورت به سطر اول برویم دیگر نمیتوان از حالت 1 خارج شد و دیگر این زنجیره مارکف ارگودیک نیست چرا که نمیتوانیم به نقطه دلخواه دیگری برویم.

پایان سوالات زنجیره مارکف

بررسی عدد π : از یک توزیع یکنواخت برای تولید اعداد درون یک مربع استفاده می کنیم. و نسبت اعداد داخل دایره را به اعداد داخل مربع می سنجیم. طبق برآورد ما عدد π از رابطه زیر به دست می آید. (نسبت نقاط داخل دایره به نقاط کل)

$$\frac{\pi r^2}{(2r)^2} = \frac{\pi}{4}$$

$$(r = 1)$$

نتایج شبیه سازی:

```
np.random.seed(0)
for i in range(1,10):
    n = 10**i
    x = np.random.uniform(-1, 1, n)
    y = np.random.uniform(-1, 1, n)
    d = np.sqrt(x**2 + y**2)
    inside = d <= 1
    pi = 4 * np.sum(inside) / n
    print("pi is",pi,"for",n,"number of points")
```

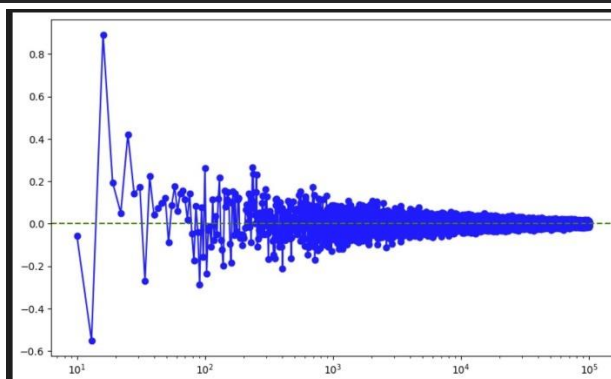
pi is 3.2 for 10 number of points
pi is 3.16 for 100 number of points
pi is 3.056 for 1000 number of points
pi is 3.1052 for 10000 number of points
pi is 3.1298 for 100000 number of points
pi is 3.141232 for 1000000 number of points
pi is 3.1411852 for 10000000 number of points
pi is 3.14145712 for 100000000 number of points
pi is 3.14167968 for 1000000000 number of points

با افزایش تعداد آزمایش ها مقدار مورد نظر به عدد π نزدیک تر می شود.

بررسی میزان خطا با افزایش تعداد آزمایش ها

```
np.random.seed(0)
errors = []
n_values = []
for i in range(10,100000,3):
    n = i
    x = np.random.uniform(-1, 1, n)
    y = np.random.uniform(-1, 1, n)
    d = np.sqrt(x**2 + y**2)
    inside = d <= 1
    pi = 4 * np.sum(inside) / n
    error = (np.pi - pi)
    errors.append(error)
    n_values.append(n)

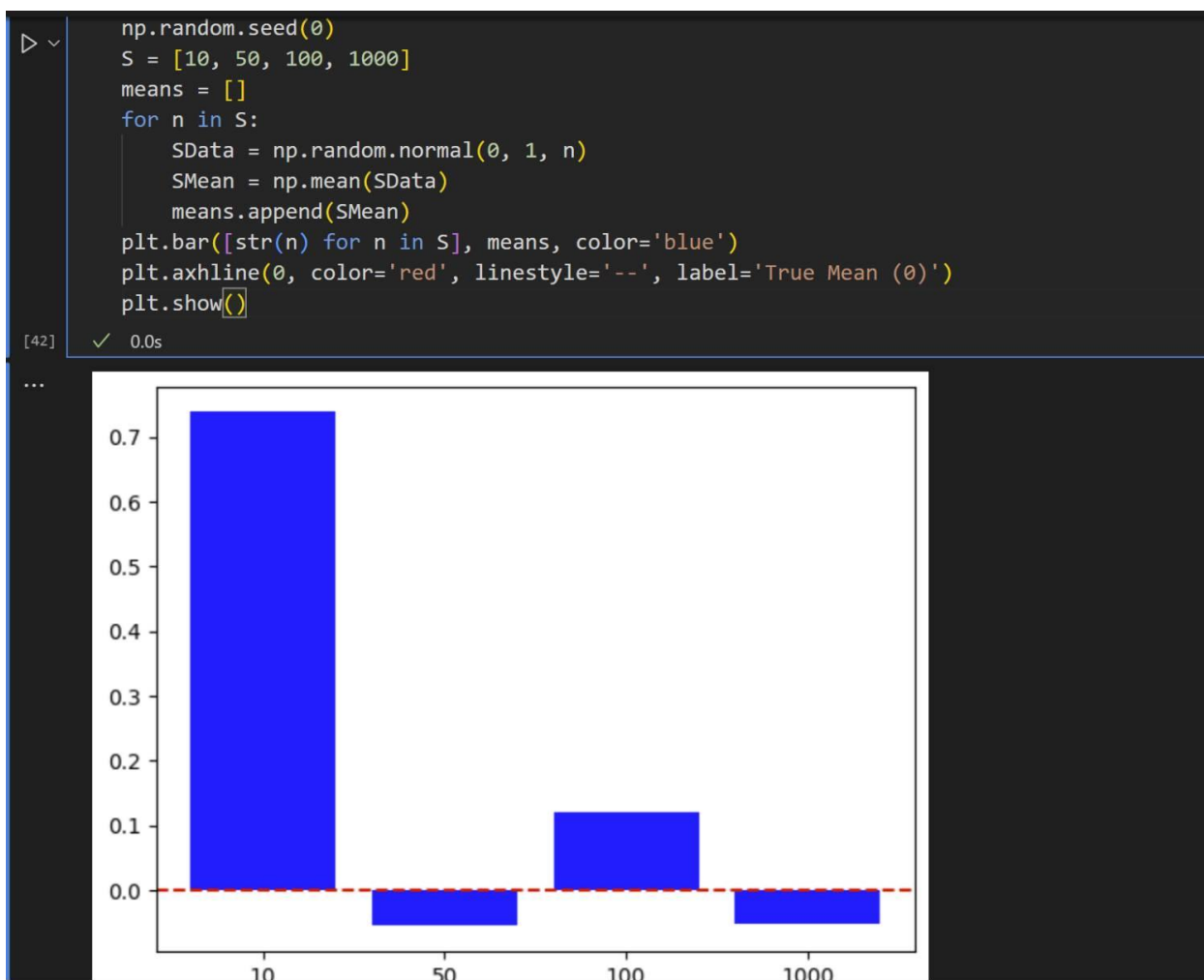
plt.figure(figsize=(10, 6))
plt.plot(n_values, errors, markers='o', linestyle='-', color='blue', label='Error in pi estimate')
plt.axhline(0, color='green', linestyle='--', label='True Mean (0)')
plt.xscale('log')
```



همانطور که در تصویر مشخص است با افزایش تعداد آزمایش ها خطای تخمین در حال کم شدن است. که این اثر قانون اعداد بزرگ است.

پرسش 2:

۲. کاربرد قانون اعداد بزرگ: یک کد پایتون بنویسید تا نمونه‌های تصادفی از یک توزیع نرمال شبیه‌سازی کنید. میانگین نمونه را برای اندازه‌های مختلف نمونه ($n = 10, 50, 100, 1000$) محاسبه کنید و نتایج را رسم کنید. چگونه میانگین نمونه به میانگین واقعی با افزایش تعداد نمونه‌ها همگرا می‌شود؟ این رفتار چگونه به قانون اعداد بزرگ (LLN) مرتبط است؟



از توزیع نرمال با میانگین صفر و واریانس 1 استفاده کردم. و میانگین داده‌ها با n های داده شده مشخص شده است. هر چه تعداد بیشتری از نمونه داشته باشیم در نهایت میانگین آن‌ها بیشتر به میانگینی که انتظار داریم توزیع مدنظر داشته باشد نزدیک‌تر است. قانون اعداد بزرگ هم درباره همین صحبت می‌کند و تضمین می‌کند که اگر تعداد داده‌ها را بی نهایت کنیم میانگین داده‌ها با میانگین مورد انتظار ما یکی خواهد شد یا به عبارتی به آن عدد همگرا خواهد شد.

۳. برآورد واریانس : از شبیه‌سازی‌های Monte Carlo برای برآورد واریانس یک متغیر تصادفی از توزیع نمایی استفاده کنید و آن را با واریانس واقعی مقایسه کنید.
چطور دقت برآورد واریانس با افزایش تعداد نمونه‌ها بهبود می‌یابد؟ این چه چیزی درباره ویژگی‌های همگرایی شبیه‌سازی‌های Monte Carlo به ما می‌گوید؟

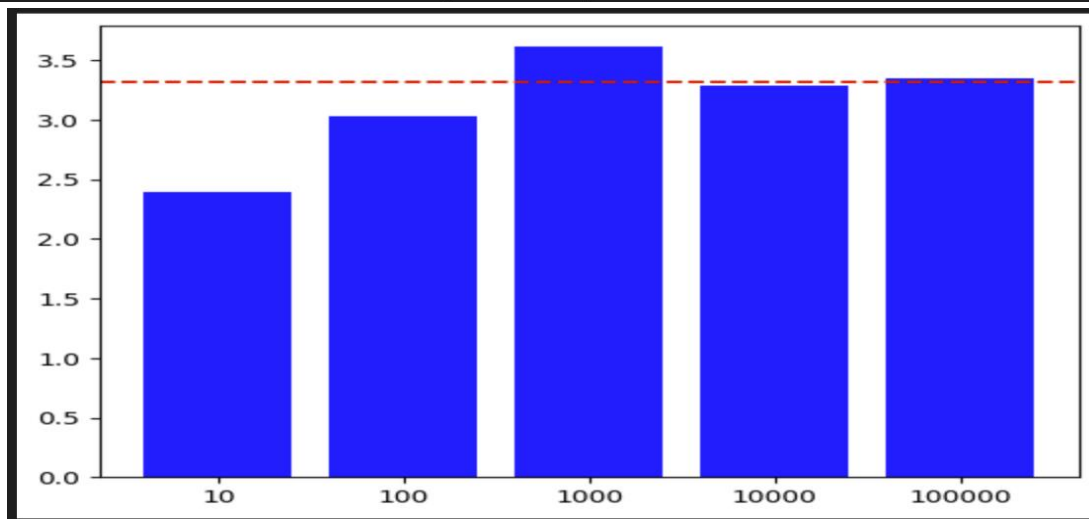
```

np.random.seed(0)
S = [10, 100, 1000, 10000, 100000]
var = []
landa = np.random.random(1)
print (landa)
for n in S:
    SData = np.random.exponential(1/landa, n)
    SVar = np.var(SData)
    var.append(SVar)
    print("for n == ", n, "difference is", SVar - 1/landa**2)
plt.bar([str(n) for n in S], var, color='blue')
plt.axhline(1/landa**2, color='red', linestyle='--', label='True Variance')
plt.show()

```

[56] ✓ 0.0s

... [0.5488135]
for n == 10 difference is [-0.92794028]
for n == 100 difference is [-0.28972806]
for n == 1000 difference is [0.29059231]
for n == 10000 difference is [-0.03408533]
for n == 100000 difference is [0.02205805]



در اینجا مقدار لاندا هم به صورت رندوم بین صفر و یک انتخاب شده و بر اساس این عدد مقادیر واریانس نمونه و واریانس احتمالی را به دست آورده شده است. و طبق عکس اول با افزایش تعداد نمونه‌ها اختلاف مقدار واریانس نمونه با واریانس این توزیع کاهش پیدا می‌کند و طبق قانون عدد بزرگ در صورت افزایش تا بی نهایت این واریانس به واریانس توزیع نمایی که برابر است با $\frac{1}{\lambda^2}$ همگرا می‌شود.

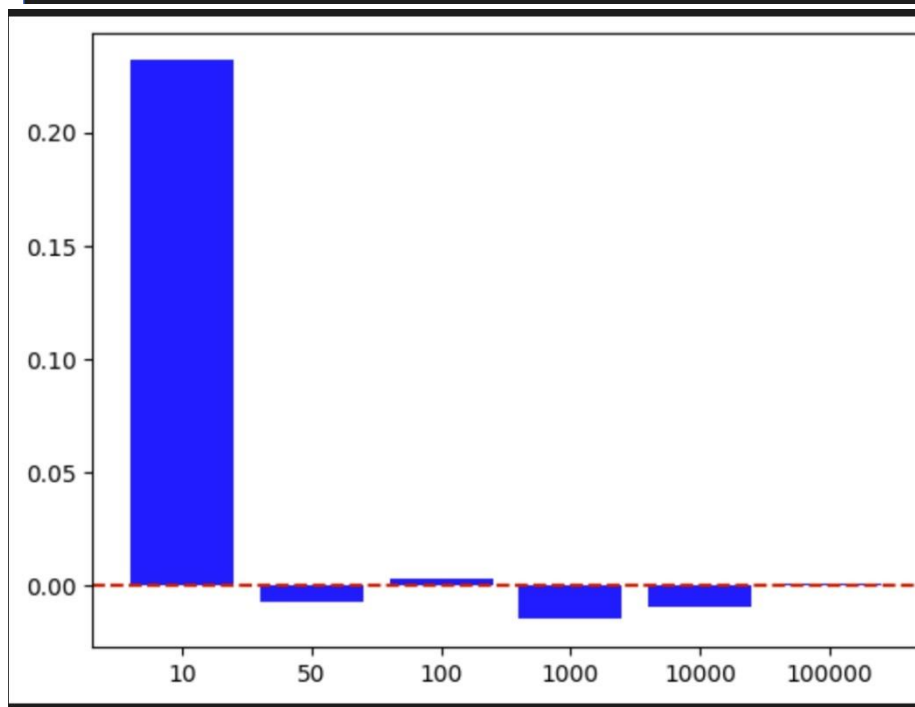
۴. CLT و همگرایی: نمونه‌های تصادفی از یک توزیع یکنواخت شبیه‌سازی کنید و میانگین نمونه را برای اندازه‌های مختلف نمونه ($n = 10, 50, 100, 1000, 10000$) محاسبه کنید. همگرایی میانگین نمونه را مشاهده کنید و توزیع میانگین نمونه‌ها را برای مقادیر مختلف n رسم کنید.

توزیع میانگین نمونه چگونه با افزایش تغییر می‌کند؟ این را به قضیه حد مرکزی (Central Limit Theorem - CLT) ربط بدهید و توضیح دهید چرا میانگین تجربی به توزیع نرمال میل می‌کند.

```
np.random.seed(0)
S = [10, 50, 100, 1000, 10000, 100000]
mean = []
for n in S:
    Sdata = np.random.uniform(-1,1,n)
    Smean = np.mean(Sdata)
    mean.append(Smean)
    print("for n == ",n,"difference is", Smean)
plt.bar([str(n) for n in S], mean, color='blue')
plt.axhline(0, color='red', linestyle='--', label='True Mean (0)')
plt.show()
```

[60] ✓ 0.0s

```
... for n == 10 difference is 0.23153256662908506
for n == 50 difference is -0.0069146967935198415
for n == 100 difference is 0.0035515101445619777
for n == 1000 difference is -0.014651669048485201
for n == 10000 difference is -0.008870852151131056
for n == 100000 difference is 0.0009205341967788099
```



در اینجا از بازه توزیع یونیفرم از -1 تا 1 انتخاب شده تا میانگین این توزیع صفر باشد. و میانگین داده‌های به دست آمده از این توزیع برای n های مختلف در تصویر های بالا نشان داده شده است.

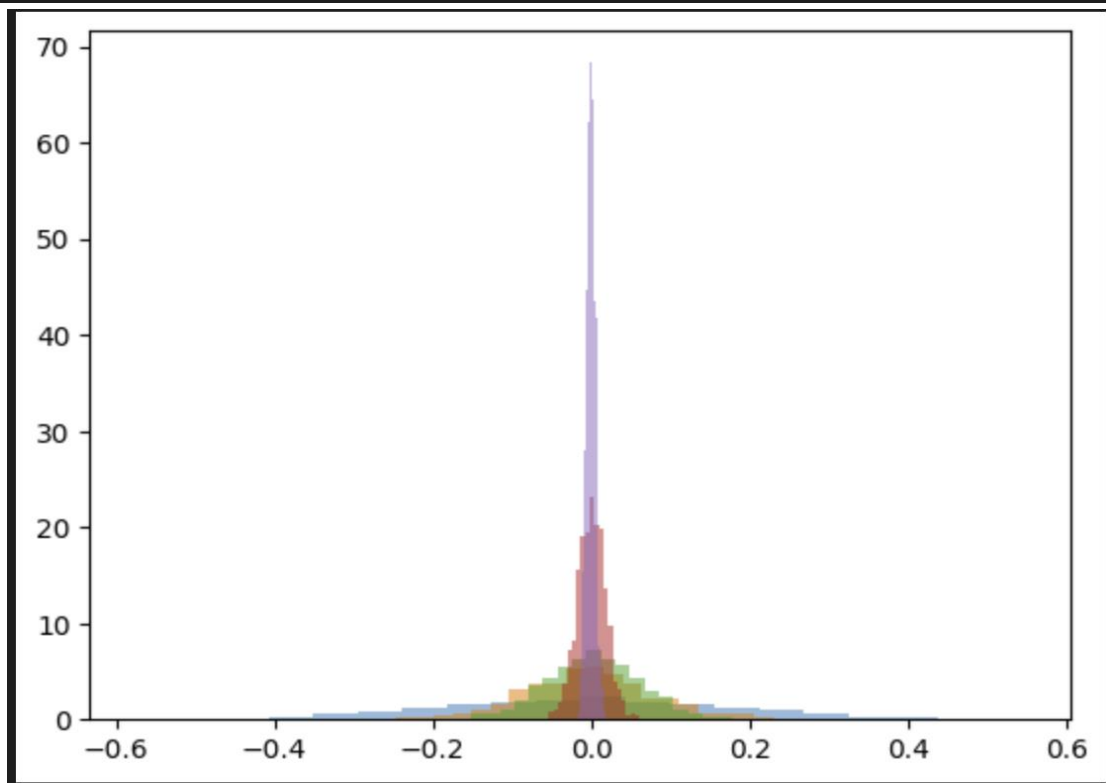
قسمت دوم این سوال: برای n های مختلف و پیدا کردن توزیع میانگین داده‌ها با n های مختلف باید برای هر n آن را تعدادی بار مشخص تکرار کنیم. برای این کار هر کدام از n ها 1000 بار تکرار شده است. و نتیجه به صورت شکل زیر است.

```
np.random.seed(0)
N = 1000
S = [10, 50, 100, 1000, 10000]

for n in S:
    means = []
    for _ in range(N):
        samples = np.random.uniform(-1, 1, n)
        means.append(np.mean(samples))
    plt.hist(means, bins=20, alpha=0.5, density=True)

plt.show()
```

[84] ✓ 0.2s



گام ها در نمودار به 20 قسمت تقسیم شده و مطابق انتظار با زیاد کردن مقدار n توزیع میانگین داده ها به توزیع گاوسی شبیه می شود. و واریانس آن کم می شود (مطابق انتظار که برابر می شود با $\frac{\sigma^2}{n}$) و میانگین به 0 که میانگین توزیع اصلی ما است بسیار نزدیک می شود.

سوال پنج:

۵. فاصله اطمینان و CLT: بعد از اجرای یک شبیه‌سازی مونت کارلو برای برآورد میانگین نمونه، یک فاصله اطمینان ۹۵٪ برای میانگین برآورد شده بسازید. چگونه قضیه حد مرکزی (CLT) استفاده از فاصله اطمینان‌های مبتنی بر توزیع نرمال را توجیه می‌کند؟ اندازه نمونه‌ها چگونه بر عرض فاصله اطمینان تأثیر می‌گذارد؟

از نامساوی چبیشف می‌توان نوشت: $E(|x_n - \mu| > \varepsilon) \leq \frac{6^2}{n\varepsilon^2}$ برای توزیع یکنواخت سوال قبلی که بین ۱- تا ۱ بود و برای $n = 100$ مقدار ε را حساب می‌کنیم.

$$\frac{1}{3 * 100 * \varepsilon^2} \leq 0 / 05 \Rightarrow \varepsilon > \frac{1}{\sqrt{15}} \approx \frac{1}{4}$$

اثر اندازه نمونه‌ها: در نامساوی بالا اگر اندازه نمونه‌ها بالاتر برود ε که در بازه $\pm \varepsilon$ در بازه برآورد مطلوب ۹۵ درصد باشد، کوچکتر پیدا می‌شود.

قضیه حد مرکزی هم برای n های بزرگ می‌تواند مورد استفاده قرار بگیرد به این صورت که اگر n بزرگ داشته باشیم به جای استفاده از نامساوی چبیشف میتوان از حدود اطمینان توزیع نرمال استفاده کرد به طوریکه ε به اندازه ۳۶ به دست می‌آید.

برای مثال برای همین توزیع و برای $n = 10000$ مقدار حاشیه ۹۵ درصد از دو طریق نامساوی چبیشف و توزیع

$$\varepsilon > \frac{1}{\sqrt{3 * 10000 * 5 / 100}} \approx 0.25 \quad 36 = \frac{\sqrt{3}}{100} \approx 0 / 17 \quad N\left(0, \frac{1}{3 * 10000}\right)$$

سوال ۶:

۶. همگرایی برای توزیع‌های مختلف: شبیه‌سازی فرآیند برآورد میانگین را برای توزیع‌های مختلف مانند نرمال، یکنواخت و توزیع Cauchy انجام دهید. میانگین نمونه را برای اندازه‌های مختلف نمونه ($n = 1000, 100, 50, 10$) محاسبه کنید و نتایج را رسم کنید.

چگونه میانگین نمونه برای توزیع‌های مختلف همگرا می‌شود؟ برای توزیع‌هایی مانند نرمال و یکنواخت، همگرایی به سرعت اتفاق می‌افتد، اما برای توزیع‌های با دنباله‌های سنگین (مثل توزیع Cauchy) این همگرایی کندتر است.


```

np.random.seed(20)
N = 10000
S = [10, 50, 100, 1000]
for i in range(3):
    plt.figure()
    for n in S:
        means1 = []
        means2 = []
        means3 = []
        for a in range(N):
            samples1 = np.random.normal(0, 1, n)
            means1.append(np.mean(samples1))
            samples2 = np.random.uniform(-1, 1, n)
            means2.append(np.mean(samples2))
            samples3 = np.random.standard_cauchy(n)
            means3.append(np.mean(samples3))
        if(i==0):
            plt.hist(means1, bins=20, alpha=0.5, density=True)
            plt.title("Normal Distribution")
            print("mean normal for n = ",n," : ",np.sum(means1)/N)
            print("mean uniform for n = ",n," : ",np.sum(means2)/N)
            print("mean cauchy for n = ",n," : ",np.sum(means3)/N)
        elif(i==1):
            plt.title("Uniform Distribution")
            plt.hist(means2, bins=20, alpha=0.5, density=True)
        else:
            plt.title("Cauchy Distribution")
            plt.hist(means3, bins=20, alpha=0.5, density=True)

plt.show()

```

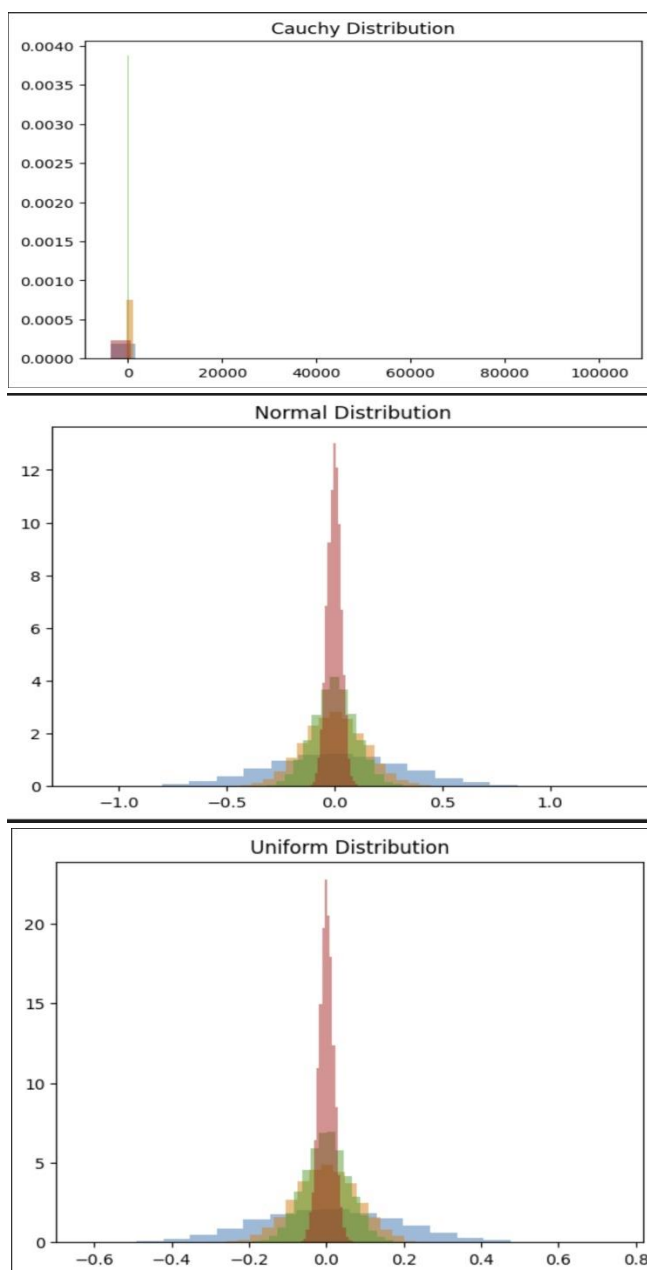
```

mean normal for n = 10 : 0.006260562289924742
mean uniform for n = 10 : 0.0006200030092755125
mean cauchy for n = 10 : 0.45604247043354607
mean normal for n = 50 : 0.0008111968578734502
mean uniform for n = 50 : 0.0004504340436209592
mean cauchy for n = 50 : 0.40089368839758904
mean normal for n = 100 : 0.0003982541554973733
mean uniform for n = 100 : 0.0012577412407513496
mean cauchy for n = 100 : 0.6661635724250047
mean normal for n = 1000 : 1.5972562448592775e-05
mean uniform for n = 1000 : -0.0001258502371169929
mean cauchy for n = 1000 : 0.5499988099741023

```

طبق قضیه حد مرکزی و اعداد بزرگ با بزرگ تر شدن تعداد نمونه ها توزیع آنها اولاً به میانگین توزیع اولیه میل میکند و همچنین توزیع نرمال دارند. در اینجا هم طبق اعداد عکس قبلی برای توزیع های نرمال و یکنواخت این همگرایی به نسبت سریع اتفاق افتاده اما برای توزیع کوشی این همگرایی با سرعت کمتری اتفاق می افتد البته که توزیع کوشی جزو توزیع هایی است که میانگین ندارد.

عکس های نمودار ها:



پرسش 7:

۷. تحلیل همگرایی برای توزیع‌های مختلف: شبیه‌سازی فرآیند برآورد میانگین را برای توزیع‌های مختلف انجام دهید و تغییرات همگرایی میانگین نمونه را بررسی کنید:

- توزیع نرمال: تولید نمونه‌های تصادفی از یک توزیع نرمال و محاسبه میانگین نمونه برای اندازه‌های مختلف ($n = 10, 50, 100, 1000$). مشاهده کنید که میانگین نمونه چگونه به میانگین واقعی (صفر برای توزیع نرمال استاندارد) همگرا می‌شود.

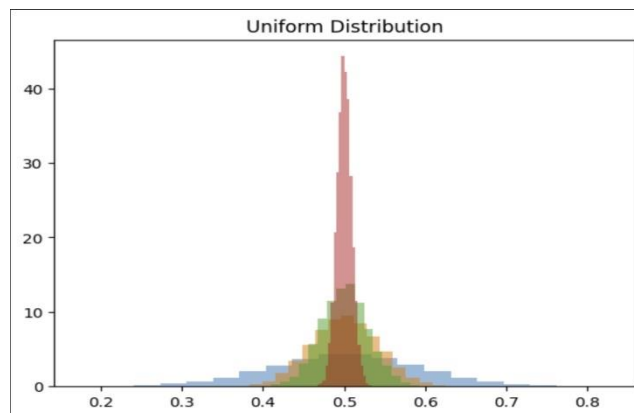
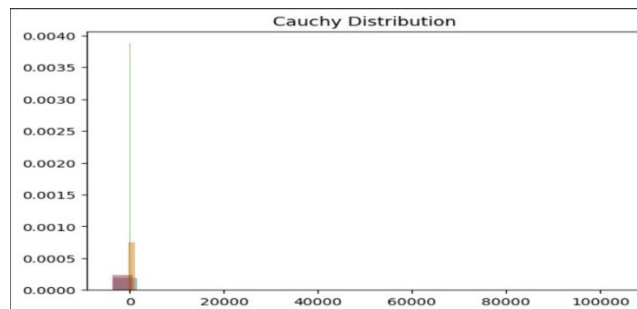
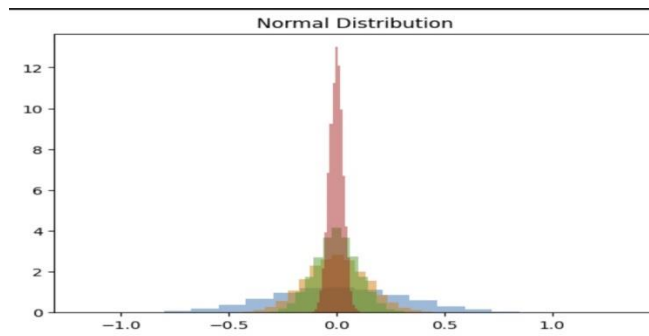
- توزیع یکنواخت: همین کار را برای توزیع یکنواخت $U(0, 1)$ انجام دهید و مشاهده کنید که میانگین نمونه چگونه به $\mu = 0.5$ همگرا می‌شود.

- توزیع Cauchy: تولید نمونه‌هایی از توزیع Cauchy و محاسبه میانگین نمونه. شما احتمالاً مشاهده خواهید کرد که میانگین نمونه حتی با اندازه‌های بزرگ نمونه‌ها هم همگرا نمی‌شود، زیرا توزیع Cauchy دارای دنباله‌های سنگین است.

پرسش این قسمت نیز مانند قسمت 6 است و نتیجه همگرایی در سوال 6 گفته شد. در اینجا صرفاً شبیه‌سازی با مقادیر گفته شده انجام شده است.

```
np.random.seed(0)
N = 10000
S = [10, 50, 100, 1000]
for i in range(3):
    plt.figure()
    for n in S:
        means1 = []
        means2 = []
        means3 = []
        for a in range(N):
            samples1 = np.random.normal(0, 1, n)
            means1.append(np.mean(samples1))
            samples2 = np.random.uniform(0, 1, n)
            means2.append(np.mean(samples2))
            samples3 = np.random.standard_cauchy(n)
            means3.append(np.mean(samples3))
        if(i==0):
            plt.hist(means1, bins=20, alpha=0.5, density=True)
            plt.title("Normal Distribution")
            print("mean normal for n = ", n, ": ", np.sum(means1)/N)
            print("mean uniform for n = ", n, ": ", np.sum(means2)/N)
            print("mean cauchy for n = ", n, ": ", np.sum(means3)/N)
        elif(i==1):
            plt.title("Uniform Distribution")
            plt.hist(means2, bins=20, alpha=0.5, density=True)
        else:
            plt.title("Cauchy Distribution")
            plt.hist(means3, bins=20, alpha=0.5, density=True)
plt.show()
```

```
mean normal for n = 10 : 0.007318396388583313
mean uniform for n = 10 : 0.499342987307402
mean cauchy for n = 10 : -0.9333156905022144
mean normal for n = 50 : 0.0018607208888914318
mean uniform for n = 50 : 0.4996531621565322
mean cauchy for n = 50 : 0.7864397992771052
mean normal for n = 100 : 0.00012142179692618704
mean uniform for n = 100 : 0.4994039892498872
mean cauchy for n = 100 : -1.1586493135519038
mean normal for n = 1000 : -6.907916262148904e-05
mean uniform for n = 1000 : 0.5000469304245362
mean cauchy for n = 1000 : 4.62053742996601
```



پرسش تئوری ۱. چند پرسش نهایی در مورد مشاهدات شما:

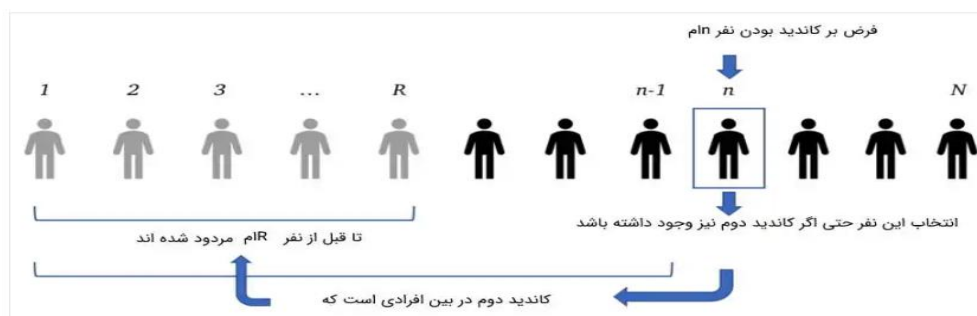
۱. چرا قضیه قانون اعداد بزرگ برای قابلیت اطمینان روش‌های Carlo Monte ضروری است؟
۲. چگونه قضیه حد مرکزی، تفسیر و اعتماد به نتایج Monte Carlo را آسان می‌کند، حتی برای توزیع‌های غیرنرمال؟
۳. آیا روش‌های Monte Carlo می‌توانند شکست بخورند؟ چه نقشی دارند مفروضات LLN و CLT (برای مثال، واریانس محدود) در تضمین موفقیت آن‌ها؟

- 1- اگر قانون اعداد بزرگ وجود نداشت تخمین زده شده از یک متغیر نامعتبر بود چرا که با زیاد کردن تکرار ها باز هم معلوم نبود که این متغیر به چه چیزی همگرا می شود و آیا با فرض همگرا شدن، به متغیر مدنظر ما همگرا می شود یا چیزی دیگر و قابل اعتماد بودن روش مونت کارلو رو ثابت می کند.
 - 2- این اتفاق باعث ساده تر شدن حدود اطمینان تخمین ما می شود و زمانی هم که با یک توزیع عجیب و ناشناخته سر و کار داشته باشیم، حدود اطمینان برای دقت جواب به دست آمده را به خوبی مشخص می کند.
 - 3- در همین جا توزیع کوشی به دلیل مشکل در واریانس تخمین ما را با مشکل مواجه کرد یا در مواردی اگر تعداد کمی نمونه گیری داشته باشیم امکان خطا و میزان خطا بالا می رود. همچنین اگر یک خطای سیستماتیک در نمونه گیری ها داشته باشیم امکان خطا بالا می رود و تخمین مونت کارلو اعتبار خودش را از دست می دهد. قانون اعداد بزرگ در شرایطی که این قضیه صدق بکند (مثل داشتن واریانس محدود) تضمین کننده درستی تخمین مونت کارلو است و قضیه حد مرکزی هم دقت این تخمین را تا حد خوبی بیان می کند.
- پایان قسمت اول

پرسش تئوری ۲. استراتژی بهینه را تحلیل کنید.

۱. احتمال انتخاب بهترین کاندید با این استراتژی را برحسب r بدست آورید.
۲. برای مقادیر بزرگ N ، این احتمال را به صورت تابعی پیوسته از r تخمین بزنید.
۳. مقدار بهینه r را برای مقادیر بزرگ N تخمین بزنید.
۴. احتمال انتخاب بهترین کاندید با استفاده از این مکانیزم چقدر است؟

با توجه به این شکل که از سایت فرادرس برداشته شده



می خواهیم احتمال انتخاب شدن بهترین نفر را برای این پست پیدا کنیم اگر r نفر را حذف کرده باشیم. اگر این فرد در جایگاه n باشد که احتمال $\frac{1}{N}$ دارد و n از r بزرگتر باشد. احتمال اینکه هیچ فردی از $r+1$ تا $n-1$ از بهترین نفر r نفر اول بهتر نباشد را باید حساب کنیم. چون در غیر اینصورت فرد غیر شایسته انتخاب می شود. که این احتمال برابر می شود با $\frac{r}{n-1}$ و برای به دست آوردن $p(R)$ باید بر روی تمام n ها سیگما بست.

$$P(r) = \sum_{n=r+1}^N \frac{r}{N} \left(\frac{1}{n-1} \right)$$

2- می توان این سری را به صورت لگاریتمی بسط داد:

$$\sum_{m=r}^{N-1} \frac{1}{m} \approx \ln(N-1) - \ln(r) = \ln\left(\frac{N-1}{r}\right)$$

با جایگزینی m با $n-1$ داریم

$$p(r) = \frac{r}{N} \ln\left(\frac{N}{r}\right)$$

3- باید مشتق این عبارت را بگیریم و مساوی صفر قرار بدهیم.

$$\frac{1}{N} \ln\left(\frac{r}{N}\right) + \frac{r}{N} \left(\frac{r}{N}\right) \left(\frac{-N}{r^2}\right) = 0$$

$$\Rightarrow \ln\left(\frac{N}{r}\right) = 1 \Rightarrow r = \frac{N}{e}$$

4- باید $p(r)$ را بر اساس عبارت بالا محاسبه کنیم.

$$p(r) = \frac{1}{e} \ln(e) \approx 0,37$$

اتمام پرسش تئوری 2

پرسش تئوری 3:

پرسش تئوری ۳. فرض کنید $D = U[a, b]$. به سوالات زیر پاسخ دهید.

۱. اگر تمامی پیشنهادها از قبل معلوم بودند، کالا به قیمت $\max_{1 \leq i \leq N} v_i$ فروخته می شد. امید ریاضی این مقدار

$\mathbb{E}[\max_{1 \leq i \leq N} v_i]$ را محاسبه کنید و آن را به عنوان یک باند بالا در نظر بگیرید.

۲. اگر فروش به صورت تصادفی انجام می شد، امید بهره چقدر بود؟ این مقدار را به عنوان یک باند پایین در نظر بگیرید.

1- باید توزیع max را پیدا کنیم و انتگرال امید ریاضی را روی آن بگیریم. از cdf استفاده می کنیم. متغیر تصادفی x را max این چند متغیر می نامیم. احتمال اینکه متغیر تصادفی X بخواهد از x کمتر شود معادل این است که تمام متغیر های v نیز از این x دلخواه کمتر شده باشند. پس cdf آن را می توان به صورت زیر نوشت:

$$F_x(x) = \left(\frac{x-a}{b-a}\right)^N$$

چون cdf توزیع یکنواخت برابر است با $\frac{x-a}{b-a}$

$$f_x(x) = \frac{N}{b-a} \left(\frac{x-a}{b-a}\right)^{N-1} \quad a < x < b$$

$$E(x) = \int_{-\infty}^{+\infty} x f_x(x)$$

با محاسبه این انتگرال به جواب : $a + (b-a) \frac{N}{N+1}$ می رسیم. که باند بالا است.

2- در این صورت امید ریاضی آن برابر می شود با امید ریاضی توزیع یکنواخت که برابر است با $\frac{b+a}{2}$

پرسش تئوری 4:

پرسش تئوری 4. فرض کنید $D = U[a, b]$. به سوالات زیر پاسخ دهید.

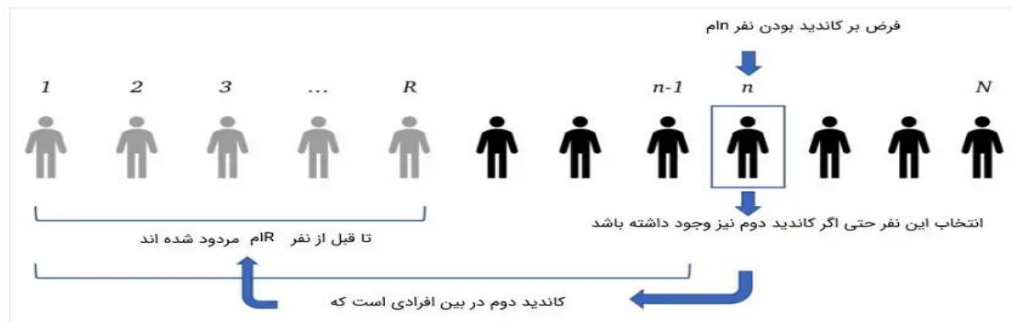
1. امید ریاضی بهره حاصل از مکانیزم توقف بهینه $E[v_{T_N}]$ را برای N های بزرگ محاسبه کنید.

2. بهره حاصل از این مکانیزم را با باند بالا و پایین مقایسه کنید.

با احتمال $p(r)$ مقدار max خود را اخذ می کند. که برابر b می شود برای n بزرگ.

مقدار دومین عدد بعد از max اگر max در جایگاه n باشد را باید حساب کنیم.

مقدار این عدد برای n بزرگ برابر همان b است. و باید احتمال آن را حساب کنیم.



یعنی در این شکل از بین نفر $r+1$ تا $n-1$ نفر دوم قرار دارد و نفر سوم در r نفر اول است.

$$P(r) = \sum_{n=r+1}^N \frac{r}{N} \left(\frac{1}{n-1}\right) * \frac{n-r-1}{n-1}$$

باید N را به اندازه خوبی بزرگ کنیم و مقدار r را برابر $\frac{N}{e}$ قرار بدهیم.

$$p(r) = \frac{r}{n} \sum_{n=r+1}^N \frac{n}{(n-1)^2} - \frac{(r+1)}{(n-1)^2}$$

چون n بزرگ است میتوان به جای $n-1$ ، n را قرار داد. و چون $r < 1$ است قسمت دوم را با بسط سری هندسی ساده کرد. قسمت اول آن نیز مثل خود مسئله منشی با استفاده از $\ln$ ساده می شود.

در نتیجه داریم :

$$p(r) = \frac{r}{N} \ln \left(\left(\frac{N}{r+1} \right) - 1 \right)$$

که به ازای n بزرگ و $r = n/e$ این مقدار برابر صفر به دست می آید. یعنی احتمال انتخاب شدن اولین عدد قبل از ماکس به صورت حدی برابر صفر است. اما ما $n-r$ عدد داریم که ممکن است هر کدام بیایند و به همین خاطر یک باند پایین برای این مقادیر که آمدن کوچکترین عدد بین این $n-r$ عدد است را انتخاب می کنیم و در احتمال آن که $1 - \frac{1}{e}$ هست ضرب می کنیم. مقدار آن برای n بزرگ برابر است با $a + (b-a) \left(\frac{r+1}{N+1} \right)$ و قرار می دهیم $r = \frac{N}{e}$ که این مقدار برابر می شود با $a + \frac{(b-a)}{e}$.

حال مقدار امیدریاضی کل را حساب می کنیم:

$$b \left(\frac{1}{e} \right) + \left(a + \frac{b-a}{e} \right) \left(1 - \frac{1}{e} \right)$$

$$\Rightarrow E(v_t) = a + \left(\frac{2}{e} - \frac{1}{e^2} \right) (b-a)$$

$$\approx a + (.735 - .135)(b-a) = a + (0/6)(b-a)$$

که به دلیل اینکه حد پایین از میانگین b ، a کمتر است. حد خوبی نیست و به جای آن همان میانگین a, b را قرار می دهیم.

$$E(V_t) = \left(a + \frac{(b-a)}{2} \right) \left(1 - \frac{1}{e} \right) + \frac{b}{e} \approx 0/68b + 0/31a = a + (0.68)(b-a)$$

برای محاسبه دقیق تر: $E(X)$

زمانی که تعداد n را زیاد کنیم max به سمت b میل می کند و میانگین هم که باند پایین است به سمت میانگین a, b .

برای این متغیر تصادفی هم با افزایش n ما با احتمال $\frac{1}{e}$ ماکس را در r تای اول داریم در نتیجه انتخاب آخر ما پاسخ خواهد بود و با احتمال $1 - \frac{1}{e}$ بیشینه در قسمت دوم خواهد بود که با افزایش n به سمت b میل خواهد کرد. (طبق قضیه ساندویچ)

$$E(v) = \frac{1}{e} \frac{(a+b)}{2} + \left(1 - \frac{1}{e} \right) (b) \approx a + 0/83(b-a)$$

مقایسه این مقدار با باند بالا و پایین نشان دهنده بالارفتن قابل توجه امید ریاضی است و محصول با قیمت بسیار مناسب تری به نسبت نصف a, b به فروش خواهد رسید اما کماکان با بیشینه مطلق که b باشد فاصله دارد.

شبیه سازی:

```
np.random.seed(0)
N = 1000
S = [10, 50, 100, 1000, 10000, 100000, 1000000]

for n in S:
    max = []
    r = int(n/np.e)
    print("r is", r)
    max1 = []
    max2 = []
    means = []
    sum = []
    x = []
    for j in range(N):
        sum1 = 0
        samples = np.random.uniform(0, 1, n)
        max.append(np.max(samples))
        max1.append(np.max(samples[:r]))
        flag = 0
        for i in range(r, n):
            if(max1[j] < samples[i]):
                flag = 1
                max2.append(samples[i])
                break
        if(flag == 0):
            sum1+=1
            max2.append(samples[n-1])
            x.append(samples[n-1])
        means.append(np.mean(samples))
        sum.append(sum1)
    print("max for n = ", n, ": ", np.mean(max))
    print("max1 for n = ", n, ": ", np.mean(max1))
    print("max2 for n = ", n, ": ", np.mean(max2))
    print("mean for n = ", n, ": ", np.mean(means))
    print("sum for n = ", n, ": ", np.mean(sum))
```

[134] ✓ 50.3s

```

r is 3
max for n = 10 : 0.9087854994283934
max1 for n = 10 : 0.7411678624073281
max2 for n = 10 : 0.7323671831623821
mean for n = 10 : 0.49645889162008944
sum for n = 10 : 0.282
r is 18
max for n = 50 : 0.9800653663347408
max1 for n = 50 : 0.9447781183863103
max2 for n = 50 : 0.8057585635795319
mean for n = 50 : 0.49856642705395127
sum for n = 50 : 0.332
r is 36
max for n = 100 : 0.9903742109644739
max1 for n = 100 : 0.9747117563910481
max2 for n = 100 : 0.8004510223212642
mean for n = 100 : 0.5014236611078418
sum for n = 100 : 0.375
r is 367
max for n = 1000 : 0.9990401125856772
max1 for n = 1000 : 0.9971807833531532
max2 for n = 1000 : 0.8078078523599143
mean for n = 1000 : 0.500389857181579
sum for n = 1000 : 0.372
r is 3678
max for n = 10000 : 0.9999004710689102
max1 for n = 10000 : 0.9997294601065648
max2 for n = 10000 : 0.8102212543333877
mean for n = 10000 : 0.499943308313268
sum for n = 10000 : 0.362

```

```

r is 3678
max for n = 10000 : 0.9999004710689102
max1 for n = 10000 : 0.9997294601065648
max2 for n = 10000 : 0.8102212543333877
mean for n = 10000 : 0.499943308313268
sum for n = 10000 : 0.362
r is 36787
max for n = 100000 : 0.9999897380785244
max1 for n = 100000 : 0.9999718852721783
max2 for n = 100000 : 0.8103390395151701
mean for n = 100000 : 0.4999665232546608
sum for n = 100000 : 0.361
r is 367879
max for n = 1000000 : 0.9999989899750145
max1 for n = 1000000 : 0.9999972516628723
max2 for n = 1000000 : 0.822546736254965
mean for n = 1000000 : 0.4999932316090574
sum for n = 1000000 : 0.363

```

همانطور که در تصاویر شبیه سازی مشاهده می کنید حدود $\frac{1}{e}$ مقدار max مطلق داخل r تای اول قرار گرفته و نمونه تصادفی آخر انتخاب شده و همچنین $max2$ که max حاصل از استفاده از مسئله منشی است با افزایش تعداد n به سمت 83 که به صورت تئوری به دست آمده بود میل می کند.

باند بالا و پایین هم طبق محاسبه تئوری به دست آمده هستند و باند بالا به سمت b که در این شبیه سازی یک است میل کرده و باند پایین هم به میانگین که $(a+b)/2$ است میل کرده است.

پایان پرسش تئوری 4 و شبیه سازی 2

پرسش تئوری 5:

Type equation here.

پرسش تئوری 5. فرض کنید $D = U[a, b]$. به سوالات زیر پاسخ دهید.

۱. امید ریاضی بهره را بر حسب τ و ϵ محاسبه کنید و نمودار آن را برای $N = 10$, $b = 1$, $a = 0$ رسم کنید.

۲. برای $N = 20$ و $\epsilon = 0.05$, مقدار τ را محاسبه کنید. امید ریاضی بهره در این حالت چقدر است؟ این مقدار را با مکانیزم‌های پیشین مقایسه کنید.

۳. رابطه‌ای برای تعیین بهترین مقدار τ در حضور N متقاضی بدست آورید.

-1

$$P(r_1 < t, v_2 < t, \dots) = \left(\frac{t-a}{b}\right)^N$$

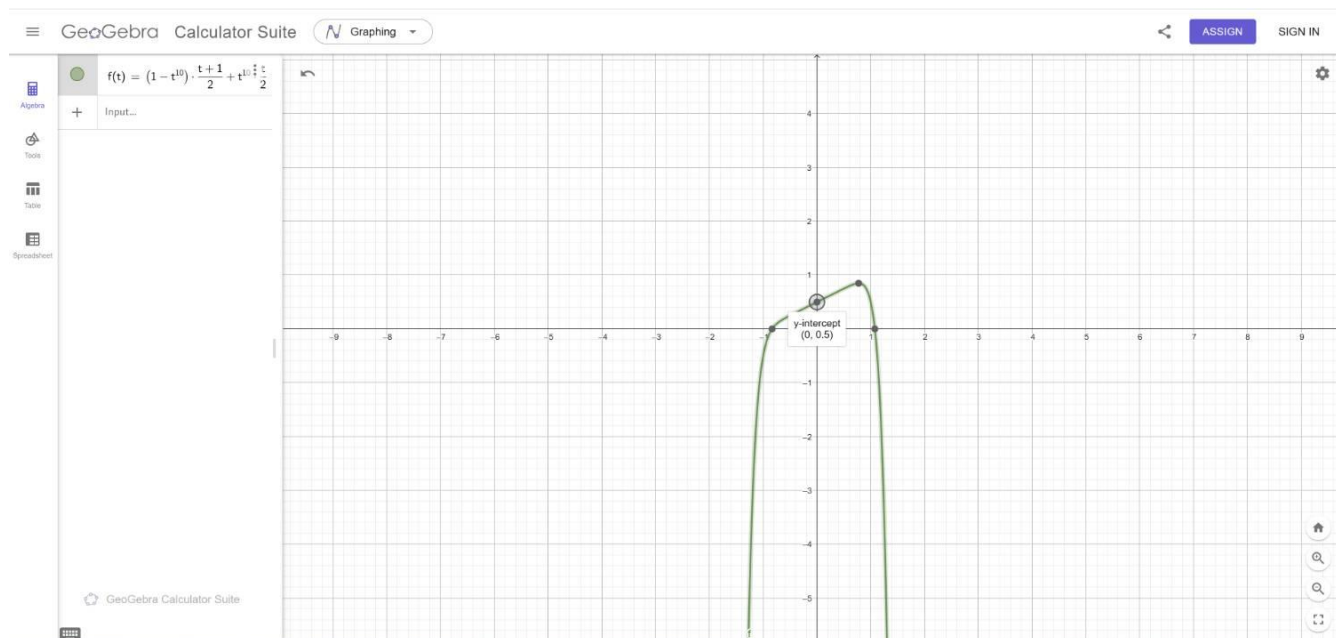
$$P(\exists i: v_i > t) = 1 - \left(\frac{t-a}{b}\right)^N$$

$$\left(1 - \left(\frac{t-a}{b}\right)^n\right) \left(\frac{t+b}{2}\right) + \left(\frac{t-a}{b}\right)^n \left(\frac{a+t}{2}\right)$$

$$\varepsilon = (t)^N$$

به ازای $n = 10$ و $a = 0$ و $b = 1$ به دست می آوریم.

$$(1 - t^{10}) \left(\frac{t+1}{2}\right) + t^{10} \left(\frac{t}{2}\right)$$



2- برای $e = 0.05$ مقدار $t = \sqrt[20]{0/05} \approx 0/86$ به دست می آید. (اگر a, b و 1 داشته باشیم).

$$\tau = (0/86)(b - a) + a$$

امید ریاضی برای این حالت برابر است با $\left(\frac{(0/86)(b-a)+a}{2}\right) + 0/05 \left(\frac{a+(0/86)(b-a)+a}{2}\right)$ که این مقدار از مقادیر به دست آمده در دفعات پیش بیشتر است.

3- باید max پیدا کنیم. یعنی از رابطه اول بر حسب t مشتق بگیریم و برابر 0 قرار دهیم تا بهترین حالت پیدا شود.

$$\left(-\frac{n}{b}\left(\frac{t-a}{b}\right)^{n-1}\right)\left(\frac{t+b}{2}\right) + \left(1 - \left(\frac{t-a}{b}\right)^n\right)\left(\frac{1}{2}\right) + \left(\frac{n}{b}\left(\frac{t-a}{b}\right)^{n-1}\right)\left(\frac{a+t}{2}\right) + \left(\frac{t-a}{b}\right)^n\left(\frac{1}{2}\right) = 0$$

پاسخ های این معادله بهینه ترین حالات هستند.

پرسش شبیه سازی:

پرسش شبیه سازی ۳. با استفاده از شبیه سازی Monte Carlo و توزیع $D = U[0, 1]$ موارد زیر را تحلیل کنید:

۱. عملکرد مکانیزم توقف بهینه برای $\epsilon = 0/05$ و مقدار بهینه ϵ^* (که با روش های عددی محاسبه می شود) را شبیه سازی کنید. عملکرد این مکانیزم را با مکانیزم های پیشین مقایسه کنید.

۲. نمودار ϵ^* و مقدار متناظر τ^* را بر حسب N رسم کنید. چه مفهومی را می توان برداشت کرد؟

بخش اول :

```

def Myfind(lst , b , n ):
    c = np.power(b, 1/n)
    for num in lst:
        if num > c:
            return num
    return lst[-1]

np.random.seed(0)
N = 1000
S = [10, 50, 100, 1000 , 10000]
e = .05
for n in S:
    means = []
    x = []
    for _ in range(N):
        samples = np.random.uniform(0 , 1, n)
        x.append(Myfind(samples,e , n))
        means.append(np.mean(samples))
    print("mean for n = ",n," : ",np.mean(means))
    print("x for n = ",n," : ",np.mean(x))

```

[26] ✓ 0.2s

```

mean for n = 10 : 0.49645889162008944
x for n = 10 : 0.8442435827499555
mean for n = 50 : 0.49856642705395127
x for n = 50 : 0.9462068174259491
mean for n = 100 : 0.5014236611078418
x for n = 100 : 0.9619040325811737
mean for n = 1000 : 0.500389857181579
x for n = 1000 : 0.9788861805335206
mean for n = 10000 : 0.499943308313268
x for n = 10000 : 0.9739988672587618

```

طبق تصاویر بالا با افزایش تعداد نمونه ها (n) مقادیر میانگین این بهره به اندازه خیلی زیادی به $\max$ نزدیک می شود. اگر $e=0.05$ ثابت استفاده شود.

قسمت دوم سوال 1:

```

def Myfind(lst, b, n):
    c = np.power(b, 1/n)
    for num in lst:
        if num > c:
            return num
    return lst[-1]

np.random.seed(0)
N = 1000
S = [10, 50, 100, 1000]
e = 0
for n in S:
    means = []
    max_mean_x = -np.inf
    e_values = np.linspace(0.01, 1, 1000)
    for e in e_values:
        x = []
        for _ in range(N):
            samples = np.random.uniform(0, 1, n)
            x.append(Myfind(samples, e, n))
        means.append(np.mean(samples))
        if np.mean(x) > max_mean_x:
            max_mean_x = np.mean(x)
            best_e = e
    print("mean for n = ", n, ": ", np.mean(means))
    print("best x for n = ", n, ": ", max_mean_x)
    print("best e for n = ", n, ": ", best_e)

```

```

mean for n = 10 : 0.4999995596966092
best x for n = 10 : 0.8585340747918241
best e for n = 10 : 0.09225225225225225
mean for n = 50 : 0.4999538696428498
best x for n = 50 : 0.9570090697156413
best e for n = 50 : 0.0318018018018018
mean for n = 100 : 0.4999903421015359
best x for n = 100 : 0.9750236506205172
best e for n = 100 : 0.01
mean for n = 1000 : 0.499992179448762
best x for n = 1000 : 0.9938833853237043
best e for n = 1000 : 0.014954954954954955

```

در اینجا هم بسته به میزان n عدد e را با استفاده از روش عددی و تغییر e و نمونه برداری با تعداد بالا به دست آورده شده است.

در صورتی که تعداد n زیاد باشد این مکانیزم بهتر از مکانیزم قبلی است چرا که میانگین خروجی های آن بسیار بالا تر است.

سوال دو:

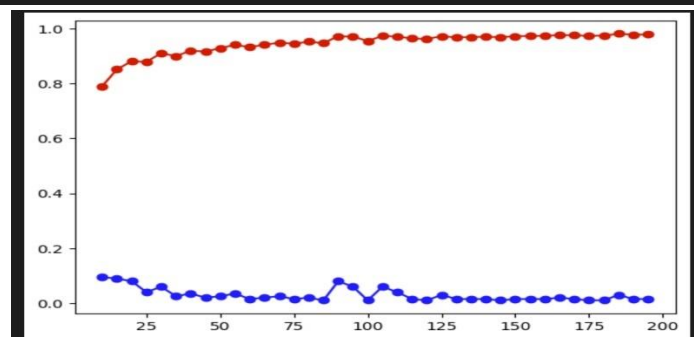
```

def Myfind(lst , b , n ):
    c = np.power(b, 1/n)
    for num in lst:
        if num > c:
            return num
    return lst[-1]

np.random.seed(0)
N = 400
S = list(range(10, 200, 5))
e = 0
beste = []
bestt = []
for n in S:
    means = []
    max_mean_x = -np.inf
    e_values = np.linspace(0.01, 1, 200)
    for e in e_values:
        x = []
        for _ in range(N):
            samples = np.random.uniform(0 , 1, n)
            x.append(Myfind(samples,e , n))
        means.append(np.mean(samples))
    if np.mean(x) > max_mean_x:
        max_mean_x = np.mean(x)
        best_e = e
    beste.append(best_e)
    bestt.append(np.power(best_e, 1/n))
    print("mean for n = ",n," : ",np.mean(means))
    print("best x for n = ",n," : ",max_mean_x)
    print("best e for n = ",n," : ",best_e)
plt.plot(S, beste, marker='o', linestyle='-', color='blue')
plt.plot(S, bestt, marker='o', linestyle='-', color='red')

```

[48] ✓ 30.6s



در اینجا هم مشخص است که با افزایش تعداد داده ها مقدار e بهینه کم شده و مقدار t افزایش پیدا می کند. (نمودار قرمز رنگ مربوط به t و آبی مربوط به e است)

یعنی با افزایش داده ها استفاده از ترشولد بالاتر بهینه تر است.

پایان پرسش تئوری 5 و شبیه سازی 3

پرسش تئوری 6:

پرسش تئوری ۶. فرض کنید $D = U[a, b]$. به سوالات زیر پاسخ دهید:

۱. برای r خریدار اول، تخمین پارامترهای D را با استفاده از تخمینگر بیشینه درست نمایی محاسبه کنید.
۲. رابطه‌ای برای تعیین T بر اساس تخمین‌های مرحله اول ارائه دهید. ابتدا برای $\epsilon = 0.05$ و سپس رابطه‌ای برای مقدار بهینه T^* بدست آورید.

- 1- از آنجا که قرار است توزیع یکنواخت را فیت کنیم. مقدار $\max$ داده‌های به دست آمده را به عنوان b و مقدار $\min$ را به عنوان a در نظر می‌گیریم. و سایر پارامترها را بر اساس توزیع یکنواخت بین a, b به دست می‌آوریم.
- 2- با چندین عدد گذاری در رابطه قسمت اول سوال 5، بستگی مقدار ترشولد ماکسیمیم و تعداد r به خوبی مشاهده می‌شود. مثلاً برای $n = 100$ ترشولد بهینه در حدود 95 فاصله است. یا برای $n = 10$ این عدد حدود 75 درصد است.
مقدار بهینه از رابطه زیر به دست می‌آید.

$$\left(-\frac{n}{b}\left(\frac{t-a}{b}\right)^{n-1}\right)\left(\frac{t+b}{2}\right) + \left(1 - \left(\frac{t-a}{b}\right)^n\right)\left(\frac{1}{2}\right) + \left(\frac{n}{b}\left(\frac{t-a}{b}\right)^{n-1}\right)\left(\frac{a+t}{2}\right) + \left(\frac{t-a}{b}\right)^n\left(\frac{1}{2}\right) = 0$$

و برای $e = 0.05$ مقدار t برابر با $\sqrt[n]{r}$ به دست می‌آید.

پرسش شبیه سازی 4:

پرسش شبیه سازی ۴. با استفاده از شبیه سازی Monte Carlo و توزیع $D = U[0, 1]$ موارد زیر را تحلیل کنید:

۱. برای مقادیر مختلف r با در نظر گیری $N = 100$ ، بهره حاصل از این مکانیزم را شبیه سازی و نمودار امید بهره مورد انتظار بر حسب r را رسم کنید.

۲. مقدار بهینه r را برای N های متفاوت را پیدا کنید و آن را رسم کنید.

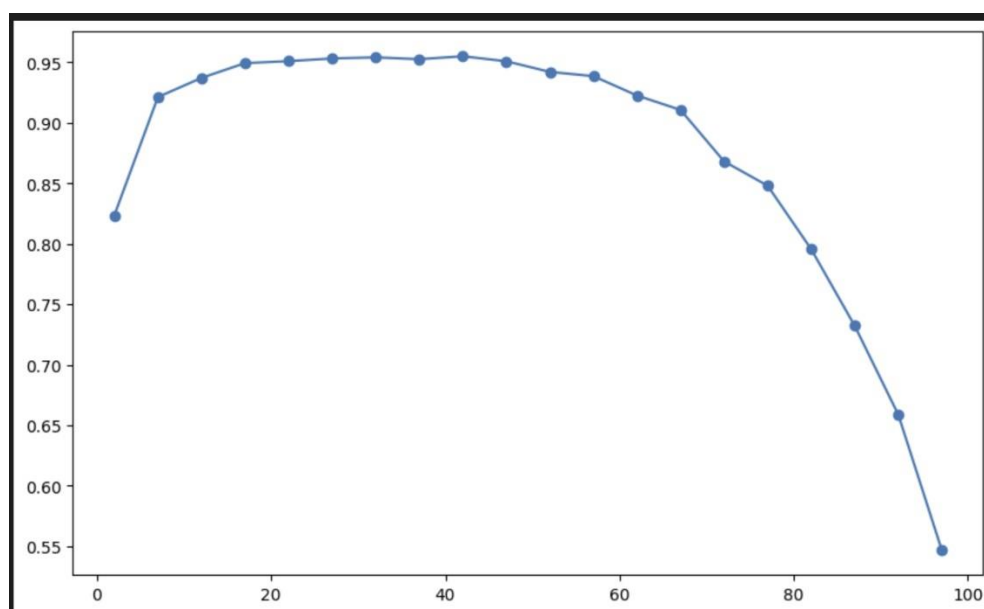
برای این سوال با توجه به $n=100$ ، مقدار t را برابر 95 قرار می دهیم. و با استفاده از تخمین انجام شده در r مرتبه اول مقدار ترشولد رو مشخص می کنیم و بر اساس آن از این بهره

```
def find_best(samples, threshold):
    for num in samples:
        if num > threshold:
            return num
    return samples[-1]

np.random.seed(0)
N = 1000
n = 100
expected_benefits = []
for r in r_values:
    selected_values = []
    for _ in range(N):
        samples = np.random.uniform(0, 1, n)
        min_val = np.min(samples[:r])
        max_val = np.max(samples[:r])
        threshold = min_val + 0.95 * (max_val - min_val)
        selected_value = find_best(samples[r:], threshold)
        selected_values.append(selected_value)
    expected_benefits.append(np.mean(selected_values))

plt.figure(figsize=(10, 6))
plt.plot(r_values, expected_benefits, marker='o', linestyle='-', color='blue')
plt.show()
```

[63] ✓ 0.2s



که نمودار آن مطابق پیش بینی به این صورت است یعنی اگر r را از یک حدی بزرگ تر کنیم یا از حدی کمتر باشد خیلی مناسب نیست.

قسمت دوم:

در این قسمت هم برای ساده تر شدن کد مقدار ترشولد رو همان مقدار 95 در نظر گرفتیم و نمودار خروجی آن به شکل زیر است:

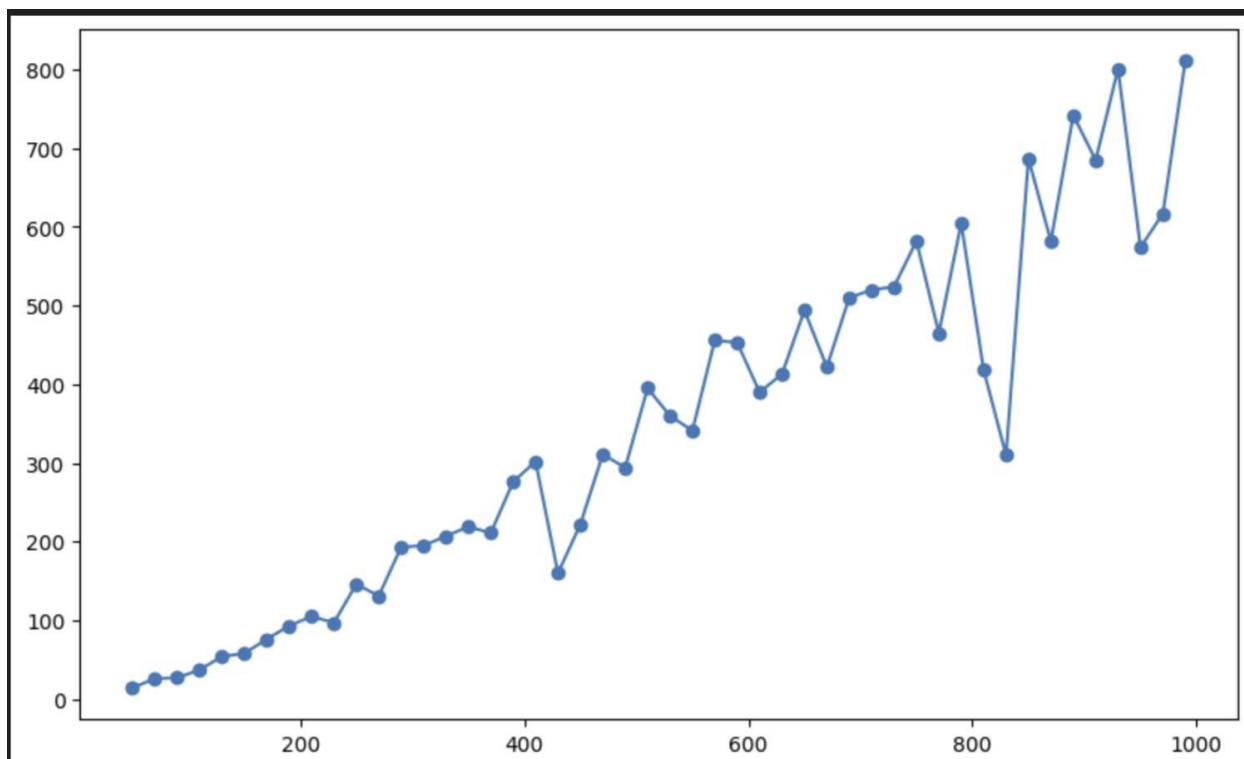
```
def find_best(samples, threshold):
    for num in samples:
        if num > threshold:
            return num
    return samples[-1]

np.random.seed(0)
N = 500
n_values = range(50, 1001, 20)
optimal_r_values = []

for n in n_values:
    max_expected_benefit = -np.inf
    best_r = None
    for r in range(2, n - 1):
        selected_values = []
        for _ in range(N):
            samples = np.random.uniform(0, 1, n)
            min_val = np.min(samples[:r])
            max_val = np.max(samples[:r])
            threshold = min_val + 0.95 * (max_val - min_val)
            selected_value = find_best(samples[:r], threshold)
            selected_values.append(selected_value)
        expected_benefit = np.mean(selected_values)
        if expected_benefit > max_expected_benefit:
            max_expected_benefit = expected_benefit
            best_r = r
    optimal_r_values.append(best_r)

plt.figure(figsize=(10, 6))
plt.plot(n_values, optimal_r_values, marker='o', label='Optimal r')
plt.show()
```

[66] ✓ 2m 20.4s



که همانطور که در تصویر مشخص است با افزایش تعداد n حالت بهینه ما برای دانستن r افزایش می یابد. حتی نسبت r به n هم افزایش می یابد به طوریکه برای $r=100$, $n=200$ و برای $r = 800$, $n=1000$ پیدا شده است.

پرسش شبیه ساز 5-

پرسش شبیه سازی ۵. مکانیزم های مختلف را با توزیع نمایی مقایسه کنید.

۱. مکانیزم مسئله منشی (توقف بهینه) را در این سناریو شبیه سازی کنید و امید بهره را بر حسب N بدست آورید.

۲. مکانیزمی که در آن ابتدا بخشی از داده ها مشاهده و سپس مقدار آستانه تنظیم می شود را بررسی کنید. امید بهره را بر اساس آستانه محاسبه کنید. فرض کنید $r = \frac{N}{p}$ باشد، میتوانید انتخاب های دیگر را نیز امتحان کنید.

۳. دو مکانیزم فوق را از نظر بهره مقایسه کنید. کدام مکانیزم عملکرد بهتری دارد؟ باند بالا و باند پایینی که در قسمت های قبل معرفی شد را نیز شبیه سازی و مقایسه کنید.

```

np.random.seed(0)
N = 1000
S = [10, 50, 100, 1000, 10000, 100000]

for n in S:
    max = []
    r = int(n/np.e)
    print("r is",r)
    max1 = []
    max2 = []
    means = []
    sum = []
    x = []
    for j in range(N):
        sum1 = 0
        samples = np.random.exponential(10, n)
        max.append(np.max(samples))
        max1.append(np.max(samples[:r]))
        flag = 0
        for i in range(r,n):
            if(max1[j] < samples[i]):
                flag = 1
                max2.append(samples[i])
                break
        if(flag == 0):
            sum1+=1
            max2.append(samples[n-1])
            x.append(samples[n-1])
            means.append(np.mean(samples))
            sum.append(sum1)
    print("max for n = ",n," : ",np.mean(max))
    print("max1 for n = ",n," : ",np.mean(max1))
    print("max2 for n = ",n," : ",np.mean(max2))
    print("mean for n = ",n," : ",np.mean(means))

```

```

r is 3
max for n = 10 : 29.107581213703064
max1 for n = 10 : 17.681320321689434
max2 for n = 10 : 18.909387821660413
mean for n = 10 : 9.907349870643644
sum for n = 10 : 0.282
r is 18
max for n = 50 : 44.91153132314274
max1 for n = 50 : 34.30376053782949
max2 for n = 50 : 29.21510283147088
mean for n = 50 : 9.94361789275085
sum for n = 50 : 0.332
r is 36
max for n = 100 : 52.05972961640627
max1 for n = 100 : 42.70043967819634
max2 for n = 100 : 32.24031902943384
mean for n = 100 : 10.053057884715441
sum for n = 100 : 0.375
r is 367
max for n = 1000 : 75.15435409392806
max1 for n = 1000 : 64.75319303252614
max2 for n = 1000 : 46.44193704186559
mean for n = 1000 : 10.009935816123388
sum for n = 1000 : 0.372
r is 3678
max for n = 10000 : 98.0969953386329
max1 for n = 10000 : 88.05406076244378
max2 for n = 10000 : 62.47154172243741
mean for n = 10000 : 9.996637471040986
sum for n = 10000 : 0.362

```

```
r is 3678
max for n = 10000 : 98.0969953386329
max1 for n = 10000 : 88.05406076244378
max2 for n = 10000 : 62.47154172243741
mean for n = 10000 : 9.996637471040986
sum for n = 10000 : 0.362
r is 36787
max for n = 100000 : 120.3875186576206
max1 for n = 100000 : 110.6550848411976
max2 for n = 100000 : 76.45804619942918
mean for n = 100000 : 9.999156975459554
sum for n = 100000 : 0.361
```

همانطور که مشخص است مقدار max2 که میانگین انتخاب های ماست بسیار از میانگین داده ها فاصله دارد.

```

def find_best(samples, threshold):
    for num in samples:
        if num > threshold:
            return num
    return samples[-1]

np.random.seed(0)
N = 1000
S = [10, 50, 100, 1000, 10000, 100000]

for n in S:
    r = n // 2
    selected_values = []
    means = []
    max_vals = []
    max1_vals = []
    max2_vals = []
    for _ in range(N):
        samples = np.random.exponential(10, n)
        max_val = np.max(samples[:r])
        min_val = np.min(samples[:r])
        threshold = min_val + 0.95 * (max_val - min_val)
        selected_value = find_best(samples[r:], threshold)
        selected_values.append(selected_value)
        means.append(np.mean(samples))
        max_vals.append(max_val)
        max1_vals.append(min_val)
        max2_vals.append(threshold)
    print("max2 for n = ", n, ": ", np.mean(max2_vals))

```

✓ 2.6s

پاسخ 2-

```

max2 for n = 10 : 21.375662607169
max2 for n = 50 : 35.662485589266346
max2 for n = 100 : 43.37196008015575
max2 for n = 1000 : 64.63231196048707
max2 for n = 10000 : 86.49802070284584
max2 for n = 100000 : 108.36863328374389

```

این مقادیر هم مقادیر میانگین به دست آمده با استفاده از یک ترشولد است.

3- به صورت کلی میانگین داده های به دست آمده از استفاده از یک ترشولد بهتر از استفاده از مسئله منشی بوده است و هر دو به صورت بسیار خوبی از مقدار میانگین که انتخاب رندم یکی از داده ها هست فاصله دارند و بهتر هستند. مقدار

میانگین برابر 10 به دست آمد. و استفاده از ترشولد فاصله کمتری تا ماکس کل داده ها که در هر تصویر نمایش داده شده دارد.

پایان
